Indian Institute of Technology Jodhpur
Department of Computer Science and Engineering

CSL3030 Operating Systems Lab

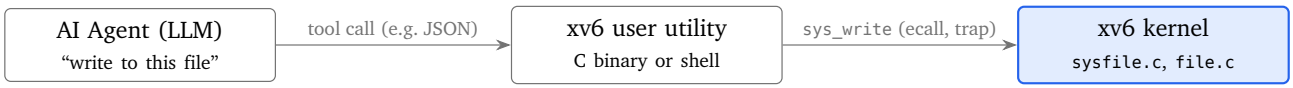
xv6 Projects

Team of 3 to 4 students · 15 to 20 days · one problem per team

Read **Before you begin** first — it contains a complete worked example of adding a system call, and the kernel patterns every problem needs. Then read your own problem, then **Evaluation rules**, **Timeline** and **Deliverables** at the end.

The code in this handout is *skeleton* code. It shows you the shape of the solution and the traps to avoid; it is not a working implementation, and filling it in is the assignment.

Every one of these projects lives at the same place in the stack. An agent decides to do something, a user-space program carries it out, and the action arrives at the kernel as an ordinary system call:



The agent can be tricked, and the user program will faithfully do whatever it is told.
The kernel is the last component that sees the action — and the only one that can refuse it.

Figure 1: Where these projects sit. Every problem in this handout adds a decision to the right-hand box.

That is the idea behind all five problems: policies an agent cannot talk its way around, because they are enforced below it.

Q	Title	Difficulty
1	Irreversibility budget for process groups	Medium
2	Deadline scheduler with priority donation through pipes	Medium
3	Taint-following capabilities	Medium–Hard
4	Exactly-once syscalls across crashes	Hard
5	Externalization barriers for speculative processes	Hard

1. Before you begin

Field	Details
Kernel	`xv6-riscv`, the tree given to you. Four things differ from older tutorials and from most code you will find online, and each one causes a confusing compile error if you copy blindly: `sleep()` is split into a prepare step followed by the sleep itself; the kernel print is `printk()`, not `cprintf()`; the log constants are `LOGBLOCKS` (30) and `MAXOPBLOCKS` (10), not `LOGSIZE`; and `copyin/copyout` take the process size, as `copyin(pagetable, sz, dst, srcva, len)`. **Always check the real prototype in `kernel/defs.h` before calling a kernel function.**
Files to read first	`syscall.c`, `syscall.h`, `sysproc.c`, `sysfile.c`, `user/user.h`, `user/usys.pl`, `proc.c`, `proc.h`, `trap.c`, `defs.h`, `param.h`. Then the files named in your own problem. Reading comes first: days 1 to 3 of the timeline are for reading, not coding.
Build and run	`make qemu` to boot, `make CPUS=1 qemu` while debugging, `make CPUS=3 qemu` to expose races, `make qemu-gdb` then attach with `riscv64-unknown-elf-gdb` and `target remote :26000`. Quit QEMU with `Ctrl-A` then `X`.
Changing `fs.h`	If you change `fs.h` or `mkfs/mkfs.c`, run `make clean && make qemu`, without exception. `mkfs` bakes the disk layout into `fs.img` when you build, so a stale `fs.img` gives you a file system that mounts happily and then returns garbage. The symptom looks nothing like the cause and students lose days to it.
Checkpoints	**Day 4:** the worked example below, running. **Day 10:** the core mechanism working for one process at `CPUS=1`, happy case only. **Day 14:** your test harness running unattended. If you are behind at day 10, cut scope that day, not at day 18.

Worked example: adding a system call, end to end Every problem starts by adding system calls, so do this one first and make sure it runs. It takes four integers, prints them, and returns their sum. There is no mechanism in it at all — it exists to prove that your six edits are correct. **This is the day 4 checkpoint.**

A system call in xv6 touches six files. Miss one and you get a link error, or worse, a program that silently is not there.

1. `user/user.h` — so user programs can call it

```
int hello(int a, int b, int c, int d);
```

2. `user/usys.pl` — generates the assembly stub that traps into the kernel

```
entry("hello");
```

3. `kernel/syscall.h` — give the call a number (next free is 23)

```
#define SYS_hello 23
```

4. `kernel/syscall.c` — declare it and put it in the dispatch table

```
extern uint64 sys_hello(void);

static uint64 (*syscalls[])(void) = {
  [SYS_fork]  sys_fork,
  /* ... the existing entries ... */
  [SYS_hello] sys_hello,     /* <-- your new entry */
};
```

5. `kernel/sysproc.c` — the implementation

```
uint64
sys_hello(void)
{
  int a, b, c, d;

  // Arguments arrive in registers a0..a5, so a syscall can take
  // at most six. argint() pulls out a 32-bit integer argument.
  argint(0, &a);
  argint(1, &b);
  argint(2, &c);
  argint(3, &d);

  printk("hello %d %d %d %d\n", a, b, c, d);
  return a + b + c + d;
}
```

6. `user/hello.c` and the `Makefile` — a program to call it

```
// user/hello.c
#include "kernel/types.h"
#include "user/user.h"

int main(void)
{
  printf("sum = %d\n", hello(1, 2, 3, 4));
  exit(0);
}
```

```
# Makefile: add to UPROGS, or the program will not exist in the xv6 shell
UPROGS=\
        $U/_cat\
        ...
        $U/_hello\
```

Now `make qemu`, type `hello` at the xv6 prompt, and you should see the kernel line and `sum = 10`. If `hello` is "not found", you missed the `Makefile`. If the link fails, you missed `usys.pl` or the table entry.

Reading other argument types. `argint(n, &i)` for 32-bit integers, `argaddr(n, &a)` for pointers *and for any 64-bit value*, `argstr(n, buf, MAXPATH)` for paths. Never dereference a pointer that came from user space directly — the user could pass any address, including a kernel one. Copy it with `copyin`/`copyout`.

Kernel patterns you will need These four patterns appear in every problem. Learn them here rather than in the middle of your mechanism.

(a) A spinlock protects shared data. Any data touched by more than one CPU needs one. Never sleep or call a function that might sleep while holding a spinlock.

```
struct spinlock lk;
initlock(&lk, "mylock");      // once, at boot

acquire(&lk);
/* ... read and write the shared data here ... */
release(&lk);                 // every path out must release
```

(b) Sleeping and waking. A process that must wait sleeps on a "channel", which is just an address used as a label. Somebody else calls `wakeup()` on the same address. **Always re-check the condition in a `while` loop, never an `if`**: by the time you wake up, another process may have taken what you were waiting for.

```
acquire(&lk);
while(!condition_is_true){     // while, NOT if
  sleep_prepare(&chan);        // check the exact prototypes in
  release(&lk);                // kernel/defs.h for your tree --
  sleep();                     // this tree splits prepare from sleep
  acquire(&lk);
}
/* condition is true and we hold the lock */
release(&lk);
```

(c) Reading the clock. `ticks` is a global counter incremented by the timer interrupt, and `clockintr()` already calls `wakeup(&ticks)` on every tick — which means sleeping on `&ticks` is a convenient way to wait for time to pass.

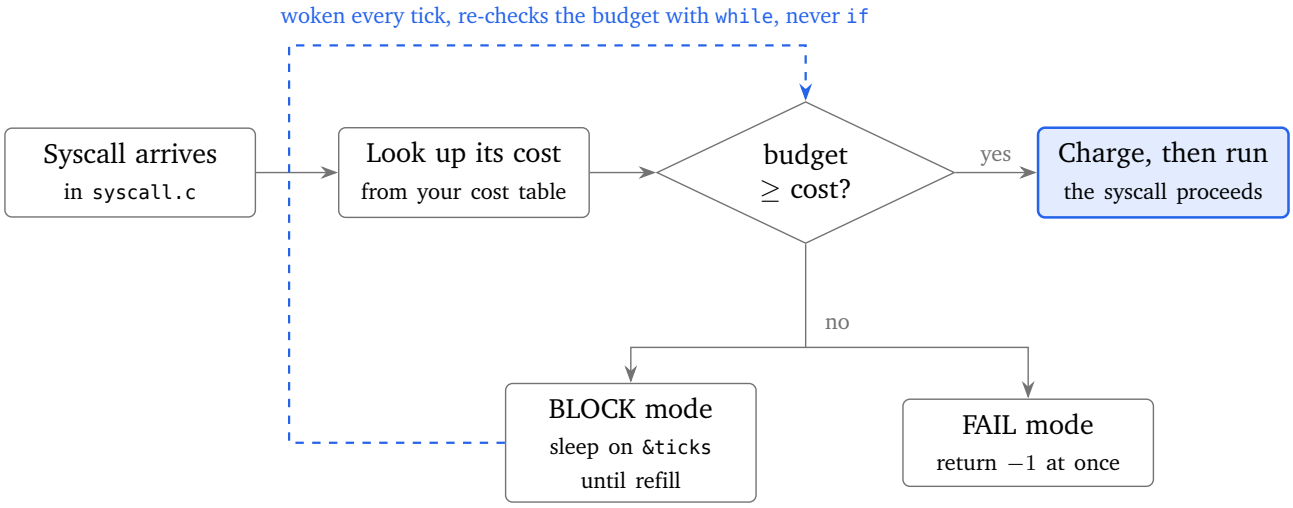
```
acquire(&tickslock);
uint now = ticks;
release(&tickslock);

// ticks is a uint and wraps around after ~49 days of ticks.
// Subtracting two uints is still correct across the wrap:
uint elapsed = now - earlier;    // correct
// int elapsed = now - earlier;  // WRONG: can go negative
```

(d) Disk transactions. Every group of related disk writes is wrapped in a transaction, so that a crash leaves the file system either fully updated or untouched — never half way. One transaction may touch at most `MAXOPBLOCKS` (10) blocks.

```
begin_op();
struct buf *b = bread(dev, blockno);
/* ... modify b->data ... */
log_write(b);        // NOT bwrite(): log_write stages it in the log
brelse(b);
end_op();            // the transaction commits here
```

2. Irreversibility budget for process groups (Medium)



Charging *after* the work leaves a budget of 1 permitting an unbounded delete: the action completes, and only then discovers it was never affordable.

Figure 2: Q1 control flow — the charge happens before the work, and block re-checks in a loop.

Field	Details
Question	Add a per-group irreversibility budget to xv6 so that a group of processes can only perform a limited amount of hard-to-undo work, and is blocked or refused once the budget runs out.
Explanation	AI agents run tools that delete files, send output to the outside world or run code written by others. Some of these actions cannot be undone. The operating system should therefore account for this risk per group and refuse new risky actions when a group has used up its allowance. xv6 today treats every syscall the same and has no notion of a group of processes or of how reversible an action is. **Why a group and not a process:** a process that is refused can simply `fork` a child, and the child would start with a fresh allowance. The budget has to belong to the whole family, or it is not a limit at all.
What to be done	**1.** Add `setgroup(gid)` and `setbudget(gid, max, refill_per_100_ticks, mode)`. A child inherits the group of its parent on `fork`. The budget is shared by the whole group, so keep it in a `groups[NGROUP]` table with one spinlock per entry, not in `struct proc`; `struct proc` gains only `int gid`. **2.** Give each syscall a cost from the cost table assigned to your team. Example costs are `unlink` 10, console write 1 per call, write to a file created by your own group 0, write to any other file 3, `exec` of a file written by another group 5. Store the creator group in the inode on disk. **3.** Charge the cost **before** the syscall does its work. The hook is the one `if` in `syscall()` that indexes `syscalls[num]`. Refill the budget from the passage of time using `ticks`, without races on `tickslock`. **4.** Mode `BLOCK` puts the caller to sleep until enough budget refills. Mode `FAIL` returns -1 at once. **5.** Write a user program `budgetstat` that prints each group, its remaining budget and the number of blocked and refused calls.

Field	Details
Files to modify	new `kernel/budget.c`, `syscall.c`, `sysproc.c`, `proc.c`, `proc.h`, `defs.h`, `trap.c`, `fs.h`, `fs.c`, `mkfs/mkfs.c`, `user/budgetstat.c`, `Makefile`.
Hints	Adding a field to `struct dinode` changes `IPB` (inodes per block), so `mkfs` and the kernel must agree — and `make clean` afterwards. Test the attack `for(;;){ if(fork()==0){ unlink("victim"); exit(0); } }` against your design: with a per-process budget it deletes without limit, which is exactly why the budget belongs to the group.
Expected output	A test `runaway` that loops over `unlink` on 200 files is stopped after the budget is used. `budgetstat` shows the exact count of calls allowed, blocked and refused, matching the cost table by hand calculation. All existing `usertests` still pass. The report has a graph of completed deletions over time for `BLOCK` and `FAIL` mode with a budget of 50. Expect `FAIL` to rise to the budget and then stay flat forever, and `BLOCK` to rise and then climb as a staircase at the refill rate.
Hidden tests	Budget inheritance across `fork` and `exec`, two groups running in parallel on 3 CPUs, and a refill happening while a process is blocked.

Code to start from `kernel/budget.c` — the group table. One lock per group, not one global lock.

```
struct group {
  struct spinlock lock;
  int used;
  int budget;          // what is left right now
  int max;             // ceiling; refills stop here
  int refill;          // units added per 100 ticks
  int mode;            // BUDGET_BLOCK or BUDGET_FAIL
  uint last_refill;    // tick count when we last refilled
  int n_allowed, n_blocked, n_refused;   // for budgetstat
};
struct group groups[NGROUP];
```

`kernel/syscall.c` — the hook. Note where the charge sits: *before* the call.

```
void
syscall(void)
{
  struct proc *p = myproc();
  int num = p->trapframe->a7;

  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {

    if(charge(p->gid, syscall_cost(num, p)) < 0) {
      p->trapframe->a0 = -1;    // refused: the work never happens
      return;
    }

    p->trapframe->a0 = syscalls[num]();   // only now does it run
  } else {
    printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

`kernel/budget.c` — lazy refill. Read the two comments carefully; both are easy to get wrong and neither shows up until you plot the graph.

```
static void
refill(struct group *g)          // caller must hold g->lock
{
  uint elapsed = ticks - g->last_refill;   // uint: survives wraparound
  int periods = elapsed / 100;

  if(periods > 0){
```

```
    g->budget += periods * g->refill;
    if(g->budget > g->max)
      g->budget = g->max;

    // Advance by whole periods only. Setting last_refill = ticks
    // throws away the remainder, and the rate drifts slow.
    g->last_refill += periods * 100;
  }
}

int
charge(int gid, int cost)
{
  struct group *g = &groups[gid];

  acquire(&g->lock);
  refill(g);

  while(g->budget < cost){          // while, NOT if
    if(g->mode == BUDGET_FAIL){
      g->n_refused++;
      release(&g->lock);
      return -1;
    }
    g->n_blocked++;

    // Sleep until the next tick, then re-check. Another process in
    // this same group may wake first and take the refilled budget,
    // which is exactly why this is a loop.
    sleep_prepare(&ticks);
    release(&g->lock);
    sleep();
    acquire(&g->lock);

    refill(g);
  }

  g->budget -= cost;
  g->n_allowed++;
  release(&g->lock);
  return 0;
}
```

3. Deadline scheduler with priority donation through pipes (Medium)

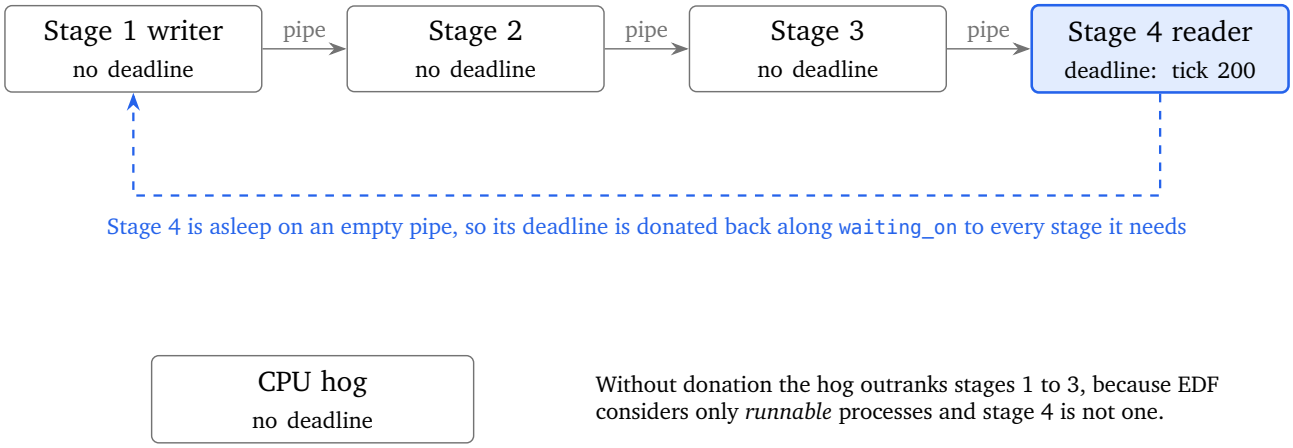


Figure 3: Q2 dependency — data flows right through the pipes, the deadline flows left.

Field	Details
Question	Replace the xv6 round-robin scheduler with a workflow-aware Earliest Deadline First scheduler in which a process waiting on a pipe lends its urgency to the process that will fill that pipe.
Explanation	An agent workflow is a chain of steps, often connected by pipes, that must finish before a deadline. If an urgent step waits for a slow helper that the scheduler treats as unimportant, the whole workflow misses its deadline. This is called *priority inversion*. xv6 records only the address a sleeper is waiting on, never *which process* is expected to wake it, so the kernel cannot see the dependency at all. Adding that one pointer is what makes the whole problem solvable.
What to be done	**1.** Add `setworkflow(wid, deadline_ticks)`. Processes in one workflow share its deadline. Processes with no workflow get a deadline of infinity. Keep the donated deadline in a field of its own, separate from the process's own deadline, so it can be withdrawn later. **2.** Change `scheduler()` in `proc.c` to always run the runnable process with the earliest effective deadline, where the effective deadline is the earlier of its own and any donated one. **3.** Record the writer side of each pipe in `struct pipe`. When a reader sleeps on an empty pipe, the writer gets the reader's deadline if it is earlier. **4.** Make donation chain through at least 4 stages and detect cycles so the kernel does not loop forever. **5.** Remove donated priority when the pipe gets data, when the reader exits, or when the donor is killed. **6.** Build a workload generator that starts pipelines with chosen lengths, CPU work per stage and deadlines.
Files to modify	`proc.c`, `proc.h`, `pipe.c`, `sysproc.c`, `syscall.c`, `syscall.h`, `defs.h`, `user/`, `Makefile`.

Field	Details
Hints	**Get EDF working and tested before you start donation.** Debugging both at once is what sinks this project. `PIPESIZE` is 512 bytes: if your stages push less than that the reader never blocks, nothing is ever donated, and your experiment measures nothing. If two readers donated to one writer, removing one donation should restore the other, not clear the field — either keep a list of donors, or recompute by scanning for any process whose `waiting_on` points here. Pick one and say why.
Expected output	A table and graph of deadline misses for round-robin, EDF without donation and EDF with donation, on 1 CPU and 3 CPUs, for 20 runs each. EDF with donation must show fewer misses on the 4-stage pipeline test. A kernel print mode shows each donation event as donor pid, receiver pid and new deadline. Round-robin should miss often; **EDF without donation should also miss often**, because the urgent process is asleep and EDF never considers it; EDF with donation should be clearly better. Know this before you plot, so you can tell a result from a bug.
Hidden tests	A 5-stage chain with an unrelated CPU hog, a pipe cycle between two processes, and a donor that exits while donation is active.

Code to start from `kernel/proc.h` — the new fields. Keep `donated` separate from `deadline`.

```
struct proc {
  /* ... existing fields ... */
  int wid;                      // workflow id, 0 = none
  uint deadline;                // its own; NO_DEADLINE = none
  uint donated;                 // lent by a waiter; NO_DEADLINE = none
  struct proc *waiting_on;      // who must run before we can
};

#define NO_DEADLINE 0xffffffff

// If you overwrite deadline with a donated value you have no way to
// restore it when the donation ends. That is why there are two fields.
static uint
effective(struct proc *p)
{
  return p->donated < p->deadline ? p->donated : p->deadline;
}
```

`kernel/proc.c` — the EDF scan. The lock discipline is the hard part.

```
struct proc *best = 0;

for(struct proc *p = proc; p < &proc[NPROC]; p++){
  acquire(&p->lock);

  // <= and not < : with <, ties always go to the lowest index in
  // proc[], and a process at a high index can starve forever.
  if(p->state == RUNNABLE && (best == 0 || effective(p) <= effective(best))){
    if(best)
      release(&best->lock);
    best = p;              // keep best->lock HELD and carry on
  } else {
    release(&p->lock);
  }
}

if(best){
  // We still hold best->lock, so it cannot have exited or slept
  // since we chose it. Releasing every lock during the scan and
  // re-acquiring the winner's afterwards is the classic bug here.
  best->state = RUNNING;
  c->proc = best;
  swtch(&c->context, &best->context);
  c->proc = 0;
```

```
  release(&best->lock);
}
```

`kernel/proc.c` — donation along the chain, with the bound that saves you.

```
void
donate(struct proc *to, uint dl)
{
  int depth = 0;

  // Two processes can genuinely block on each other's pipes, making
  // this chain a cycle. Without the bound the kernel spins here with
  // interrupts off and the machine hangs unrecoverably.
  while(to != 0 && depth++ < NPROC){
    if(to->donated <= dl)
      break;                 // already at least this urgent: stop
    to->donated = dl;
    to = to->waiting_on;     // follow the chain backwards
  }
}
```

`kernel/pipe.c` — recording the dependency just before sleeping.

```
// in piperead(), where the reader finds the pipe empty:
while(pi->nread == pi->nwrite && pi->writeopen){
  struct proc *p = myproc();

  p->waiting_on = pi->writer;             // who we are waiting for
  if(pi->writer)
    donate(pi->writer, effective(p));     // lend our urgency

  sleep_prepare(&pi->nread);
  release(&pi->lock);
  sleep();
  acquire(&pi->lock);
}
myproc()->waiting_on = 0;                 // and undo it on the way out
```

4. Taint-following capabilities (Medium–Hard)

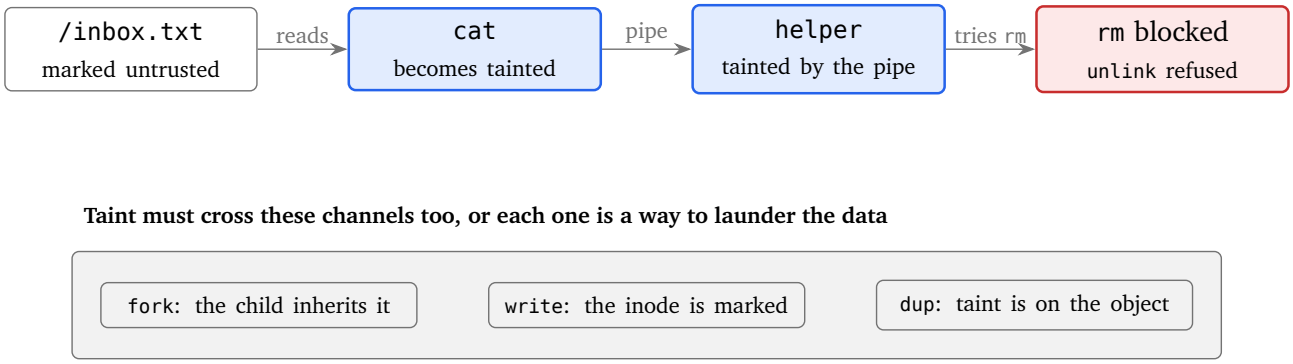


Figure 4: Q3 the basic attack — and the three other channels taint has to cross.

Field	Details
Question	Implement kernel taint tracking in xv6 so that a process which has read untrusted data loses dangerous permissions, and the taint follows the data through `fork`, pipes and files.
Explanation	An agent that reads a web page or an email can be tricked by text hidden in it, because to the model instructions and data are the same words. What an agent is permitted to do should therefore depend on where its input came from, not on who it is. If a process has read untrusted content, it should not be able to delete files or run programs. The hard part is *laundering*: data can be passed through other processes and files to hide where it came from, and `cat /inbox.txt | helper` leaves `helper` perfectly clean unless taint crosses the pipe.
What to be done	**1.** Add an untrusted flag to `struct dinode` and a syscall `marktaint(path)`. Update `mkfs` so the disk layout still works. **2.** A process that reads an untrusted file becomes tainted; `struct proc` gains `int tainted`, which is never cleared once set — bytes cannot be un-read. **3.** Spread taint through `fork`, through files a tainted process writes (they become untrusted on disk) and through pipes, so a reader becomes tainted when it reads bytes written by a tainted writer. Choose per-pipe or per-byte-range tracking and justify the choice. **4.** A tainted process cannot call `unlink`, cannot `exec` any file outside `/safe` and cannot write to a file that is not untrusted. Such calls return -1 and log one kernel line. **5.** Add `istainted()` for tests.
Files to modify	`fs.h`, `fs.c`, `mkfs/mkfs.c`, `sysfile.c`, `file.c`, `pipe.c`, `exec.c`, `proc.c`, `proc.h`, `syscall.c`, `syscall.h`, `defs.h`, `Makefile`.

Field	Details
Hints	Taint survives `exec`: the program image changes, the process does not. Clear the flag when an inode is reused in `ialloc`, or a deleted-and-recreated filename launders the data. Per-pipe tracking is about five lines but over-broad; per-byte-range is precise but the ring buffer wraps modulo `PIPESIZE` and your ranges must wrap with it. **Per-pipe plus a test that demonstrates the false positive scores better than a half-finished per-byte attempt.** Rule 4's third clause is a "write-down" rule that stops laundering into a clean file — do not implement it as "a tainted process cannot write at all", which breaks `cat` and will show up in your false-positive table.
Expected output	The demo marks `/inbox.txt` untrusted, runs `cat /inbox.txt` into a pipe into a second program, and shows the second program is refused when it tries `rm`. A report lists at least 6 laundering attacks the team tried, such as tainted to pipe to clean helper to file, and shows each one blocked, naming the mechanism that blocked it. A false-positive table shows which standard xv6 user programs break under the policy and why, and whether each breakage is a bug in your implementation or a true consequence of the policy. Distinguishing those two is the point.
Hidden tests	Laundering through `dup`, through a file written then deleted then recreated, and through a pipe shared by one tainted and one clean writer.

Code to start from `kernel/fs.h` — where to put the flag. `struct dinode` is *exactly* 64 bytes today, which is why `IPB` is 16. Pick one option and justify it in your report.

```
// Today: 4 shorts (8) + 1 uint (4) + addrs[NDIRECT+1] = 13*4 (52) = 64
struct dinode {
  short type, major, minor, nlink;
  uint  size;
  uint  addrs[NDIRECT+1];
};

// Option A: steal a bit from type. Size and IPB unchanged, but every
//           read of type now needs masking, everywhere.
// Option B: add "short untrusted;". Struct becomes 68, IPB drops to
//           15, and mkfs MUST be changed to match.
// Option C: shrink NDIRECT 12 -> 11 and use the freed slot. Size and
//           IPB unchanged; maximum file size falls.
```

`kernel/pipe.c` — taint on the *object*. Putting it on `struct file` instead is the single most common way to fail this project, because `dup` then defeats the whole policy.

```
struct pipe {
  /* ... existing fields ... */
  int tainted;        // on the pipe itself, not on struct file
};

// in pipewrite():
if(myproc()->tainted)
  pi->tainted = 1;

// in piperead(), after bytes have actually been copied out:
if(pi->tainted)
  myproc()->tainted = 1;
```

`kernel/sysfile.c` — a restriction, and the one line of logging it must print.

```
uint64
sys_unlink(void)
{
  struct proc *p = myproc();
```

```
  if(p->tainted){
    printk("taint: unlink refused for pid %d\n", p->pid);
    return -1;
  }
  /* ... the existing implementation ... */
}
```

5. Exactly-once syscalls across crashes (Hard)

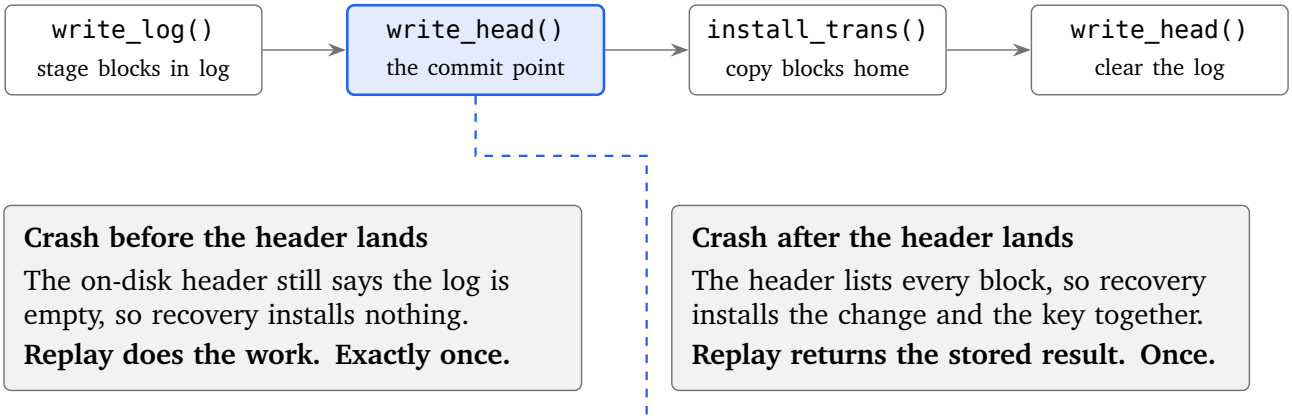


Figure 5: Q4 `commit()` in `log.c` — where a crash lands, and what the replay then does. A single block write is atomic on the disk, so there is no third case.

Field	Details
Question	Add an idempotent `effect` syscall to xv6 that guarantees a file effect happens exactly once, even if the machine crashes and the same request is replayed after reboot.
Explanation	Consider an agent that placed an order, timed out after the order had already gone through, and retried. Two orders shipped. Nothing there was buggy in the ordinary sense: retrying a lost reply is correct, and carrying out what you receive is correct. What was missing is a way to say "this is the same request I already handled" — an *idempotency key*. The OS remembers which keys it already completed and returns the old result instead of doing the work again. This only works if the record of the key and the effect itself are saved together, so a crash can never leave one without the other. xv6 already groups disk writes into transactions in `log.c`, and this project must reuse that.
What to be done	**1.** Add `effect(key, op, path, buf, n)` where `op` is `APPEND` or `CREATE` and `key` is a 64-bit number. **2.** Keep an on-disk effect table that maps key to result, stored in reserved blocks set up by `mkfs`; record their location in the superblock. **3.** Write the table entry and the file change inside the same `begin_op` and `end_op` transaction. **4.** On replay with a key already in the table, return the stored result and change nothing. **5.** Add a kernel crash flag `crashat(n)` that calls `panic` at the nth `log_write`, so crashes can be injected at every point. **6.** Write a QEMU script that crashes, reboots, replays the same request and checks the file.
Files to modify	new `kernel/effect.c`, `fs.h`, `fs.c`, `log.c`, `mkfs/mkfs.c`, `sysproc.c`, `syscall.c`, `syscall.h`, `defs.h`, a shell script in the repository root, `Makefile`.

Field	Details
Hints	**Understand `commit()` before writing anything.** It runs `write_log()`, then `write_head()`, then `install_trans(0)`, then clears the header. `write_head()` is the only atomic moment in the file system: before it completes, the header says the log is empty and recovery installs nothing; after it completes, the header lists every block and recovery installs all of them even if the machine dies during the copy. That is the whole reason one transaction covering both the key and the file change is sufficient. An entry of `{uint64 key; int result; uint pad;}` is 16 bytes, so 64 entries fit in a block; hash the key modulo the table size with linear probing. Your harness must **not** rebuild `fs.img` between runs, or recovery has nothing to recover and your results prove nothing. `MAXOPBLOCKS` is 10 and one append already touches the inode block, a bitmap block, data blocks and now a table block — keep payloads small, or raise the limit and say so, but never call `end_op()` early and reopen a transaction to get around it.
Expected output	For every crash point from 1 to the last `log_write` of an effect, after reboot and replay the file holds the appended data exactly once, never zero times and never twice. A table lists each crash point and the observed result. The report includes a short argument for why each crash point is safe, and the cost in extra disk writes per effect.
Hidden tests	Two processes using the same key at the same time, a key reused with a different payload, and an effect table that fills up.

Code to start from `kernel/effect.c` — the shape of the whole solution. Everything that matters is the fact that the lookup, the work and the insert sit inside *one* `begin_op/end_op` pair.

```
uint64
sys_effect(void)
{
  uint64 key, ubuf;
  int op, n, res;
  char path[MAXPATH];

  // 64-bit value: argaddr, NEVER argint. argint truncates to 32 bits,
  // so two keys differing only above bit 31 would collide silently.
  argaddr(0, &key);
  argint (1, &op);
  argstr (2, path, MAXPATH);
  argaddr(3, &ubuf);
  argint (4, &n);

  // Two processes with the same key can both miss the table and both
  // do the work, because nothing is on disk until commit. An
  // in-memory lock keyed on the key closes that window.
  key_lock(key);

  begin_op();                          // ONE transaction for both

  if(effect_lookup(key, &res) == 0){   // already completed
    end_op();
    key_unlock(key);
    return res;                        // return the stored result
  }

  res = do_append(path, ubuf, n);      // the effect itself
  effect_insert(key, res);             // and the record of it

  end_op();                            // both become durable here
  key_unlock(key);
  return res;
}
```

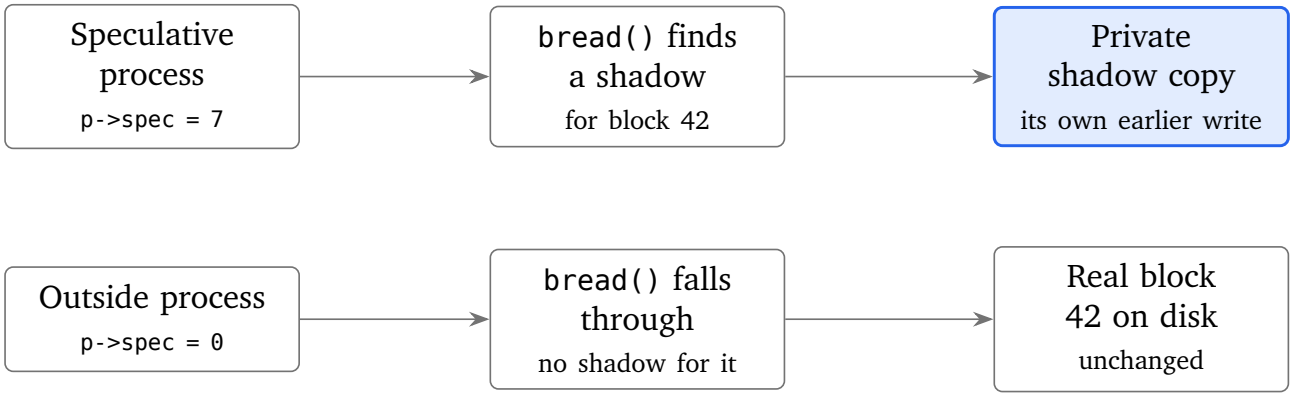
`kernel/log.c` — crash injection. This hook only reaches crashes while blocks are being staged; add a second one inside `commit()` to reach the interesting ones.

```
int crashpoint = 0;     // set by the crashat() syscall; 0 = disabled
int writecount = 0;

void
log_write(struct buf *b)
{
  if(crashpoint && ++writecount >= crashpoint)
    panic("crashat: injected crash");

  /* ... the existing implementation ... */
}
```

6. Externalization barriers for speculative processes (Hard)



On `xcommit` every shadow block is written home in one log transaction, and only after `end_op()` returns is the held console output released. On `xabort` the slots are freed and nothing was ever real.

Figure 6: Q5 the same `bread()` call from two processes — isolation is one check on the `spec` field.

Field	Details
Question	Let a process and all its children run in a speculative mode where every file write and every output to the outside is held back, then made visible all at once on commit or thrown away on abort.
Explanation	Agents often explore. They try a command, look at the result and decide whether to keep it. If that command deleted a file or printed to a user, the effect is already out in the world. *Externalization* is the moment an effect becomes visible to anyone other than the process that produced it; an effect not yet externalized can still be withdrawn. In xv6 that means file writes, creates and deletes, console output, and pipe writes to readers outside the speculation. The one thing that is *not* externalization is reading your own write — and because the speculative process must still see its own changes, a simple "queue the writes for later" design will not work. You need a shadow copy.
What to be done	**1.** Add `xbegin()`, `xcommit()` and `xabort()`. A process tree created after `xbegin` shares one speculation; `struct proc` gains `int spec` and `fork()` copies it. **2.** Redirect writes from the speculation to shadow blocks. Choose to shadow at the buffer cache level or the inode level and justify it. Reads inside the speculation must see the shadow. Reads outside must see the real disk. **3.** Buffer console output and pipe writes to processes outside the speculation until commit. **4.** Handle `create` and `unlink` inside the speculation, so a file deleted speculatively still exists for everyone else. **5.** On `xcommit`, apply all shadow changes in one log transaction. On `xabort`, free everything. **6.** Define and implement a rule for a conflict, when an outside process changes a file the speculation also changed.
Files to modify	`bio.c`, `log.c`, `fs.c`, `file.c`, `pipe.c`, `console.c`, `proc.c`, `proc.h`, `sysproc.c`, `sysfile.c`, `syscall.c`, `syscall.h`, `defs.h`, `param.h`, `Makefile`.

Field	Details
Hints	**This is the hardest of the five.** A working `xbegin`/`xabort` for file writes alone, measured and honestly documented, is a good project; attempting all six steps badly is not. If you are behind at day 10, drop step 6 first. Shadowing at the buffer cache is the recommended choice: inodes, directories and the bitmap are all just blocks, so `create` and `unlink` work for free, at the cost of a map lookup on every `bread` in the system. Running out of shadow slots must fail the write with -1, because falling back to the real disk would externalize. For step 3 the test is not "am I speculative" but "is my reader outside my speculation" — buffering a pipe write to a reader inside the same speculation breaks every pipeline and looks exactly like a hang. Flush the held output only **after** `end_op()` returns, or a crash in between prints something that then did not happen. `MAXOPBLOCKS` is 10 and `LOGBLOCKS` is 30, so a speculation touching more than ten blocks cannot commit in one transaction: raise the limits, commit in several transactions and lose atomicity, or cap the speculation size. All three are defensible; not noticing is not. For conflicts the three real options are to abort the speculation, to let the commit win, or to abort only if the speculation also *read* that block before the outside write — the last needs a read set and a per-block version counter, which is how hardware transactional memory does it.
Expected output	A speculative child runs `rm a` and `echo hi` into `b`, then aborts, and the real file system and console show no trace. The same run with commit shows both effects at once. A conflict test shows the team's chosen rule in action. A graph shows write cost inside and outside a speculation for file sizes 1 KB to 64 KB, including the cliff where the shadow table runs out — show the cliff rather than cropping it.
Hidden tests	Nested `xbegin`, a crash during `xcommit`, and a speculation that runs out of shadow blocks.

Code to start from `kernel/param.h` and `kernel/bio.c` — a fixed shadow table is enough. At `NSHADOW` 64 this is 64 KB of kernel memory.

```
#define NSHADOW 64

struct shadow {
  int  spec;            // which speculation owns this copy; 0 = free
  uint blockno;         // which block it shadows
  char data[BSIZE];     // the private contents
};
struct shadow shadows[NSHADOW];
```

`kernel/bio.c` — the redirect. This one check is the entire isolation property: a process outside has `spec == 0` and falls straight through to the normal path.

```
struct buf*
bread(uint dev, uint blockno)
{
  struct proc *p = myproc();

  if(p && p->spec){
    struct shadow *s = shadow_find(p->spec, blockno);
    if(s)
      return shadow_buf(s);      // our own earlier write
  }

  /* ... the existing buffer-cache path, untouched ... */
}

// On a write inside a speculation, allocate a shadow slot instead of
// touching the real block. If no slot is free you MUST fail with -1:
```

```
// falling back to the real disk would externalize, which is the one
// thing this whole mechanism exists to prevent.
```

7. Evaluation rules

No project is fully LLM-proof, so these rules carry as much weight as the problems themselves.

Rule	How it works
Per-team parameters	Each team gets its own cost table, deadline formula or conflict rule. A generic answer then fails the hidden tests. Read your parameters from one place that is easy to change, never written into the logic.
Hidden tests	You get about 30% of the tests. The crash, race and laundering tests stay with the instructor. The categories are listed on your problem, so you know what to defend against.
Own run evidence	Graphs must come with the raw QEMU logs that produced them. One GDB session log showing a real bug you found and fixed is required.
Live viva change	In the viva each team makes a small change on the spot, for example extend taint through `dup`, or move the crash point inside `install_trans`.
Design journal	A git commit with a short note every 2 days on what failed and why. One huge final commit is a red flag.
Documented limits	A limitation you found and wrote down scores above one you concealed. State what your model leaves out compared with a full implementation.

8. Timeline

Days	Work
1 to 3	Read the problem statement and the relevant xv6 files
4 to 10	Build the core mechanism
11 to 14	Write tests and the crash or race harness
15 to 17	Run measurements and make graphs
18 to 20	Write the report and prepare for the viva

9. Deliverables

1. **The zip file**, named `Team_Name_problem_no`, containing *only* the files that are added or modified for the implementation. Per person on the team, highlighting their part of the work.

2. **A handwritten report of 2 pages**, per team, explaining how you solved the problem and the problems encountered, well explained with mathematical expressions if required. **Strictly 2 pages.**